

Docker

Complete Guide – From Scratch

Introduction • Architecture • Commands Reference

Images & Dockerfile • Docker Compose • Storage & Volumes

Networking • Multi-Stage Builds • Best Practices • Security

Clear Definitions • Architecture Diagrams • Real-World Examples

Table of Contents

1. Introduction to Docker

- [1.1 What is Docker? The Container Revolution](#)
- [1.2 VMs vs Containers – Key Differences](#)
- [1.3 Docker Architecture – Engine, Daemon, Client, Registry](#)
- [1.4 Core Concepts – Image, Container, Registry, Volume, Network](#)
- [1.5 Installing Docker](#)
- [1.6 Docker Hello World](#)

2. Docker Commands – Complete Reference

- [2.1 Container Lifecycle Commands](#)
- [2.2 Inspecting & Monitoring Containers](#)
- [2.3 Executing Commands Inside Containers](#)
- [2.4 Copying Files & Logs](#)
- [2.5 Image Commands](#)
- [2.6 System & Cleanup Commands](#)

3. Docker Images & Dockerfile

- [3.1 What is a Docker Image? Layers Explained](#)
- [3.2 Dockerfile Instructions – Complete Reference](#)
- [3.3 Building Images – docker build](#)
- [3.4 Multi-Stage Builds](#)
- [3.5 .dockerignore](#)
- [3.6 Image Tagging & Pushing to Registry](#)
- [3.7 Dockerfile Best Practices](#)

4. Docker Compose

- [4.1 What is Docker Compose?](#)
- [4.2 compose.yml Structure & Reference](#)
- [4.3 Services, Networks & Volumes](#)
- [4.4 Environment Variables & Secrets](#)
- [4.5 Dependency & Health Checks](#)
- [4.6 Compose Commands](#)
- [4.7 Full Spring Boot + PostgreSQL + Redis Example](#)
- [4.8 Compose Profiles & Override Files](#)

5. Docker Storage & Volumes

- [5.1 Container Filesystem & Layers](#)
- [5.2 Storage Types – Volumes, Bind Mounts, tmpfs](#)
- [5.3 Volume Commands & Management](#)
- [5.4 Volumes in docker run](#)
- [5.5 Volumes in Docker Compose](#)
- [5.6 Backing Up & Restoring Volumes](#)
- [5.7 Storage Drivers](#)

6. Docker Networking

- [6.1 Docker Networking Architecture](#)
- [6.2 Network Drivers – bridge, host, overlay, none](#)

- 6.3 Default Bridge Network
- 6.4 User-Defined Bridge Networks
- 6.5 Port Publishing
- 6.6 Container DNS & Service Discovery
- 6.7 Overlay Networks for Swarm / Multi-Host
- 6.8 Network Commands

7. Docker Registry

- 7.1 Docker Hub
- 7.2 Private Registry
- 7.3 Amazon ECR, GCR, GHCR
- 7.4 Image Scanning & Security

8. Docker Security & Best Practices

- 8.1 Running as Non-Root User
- 8.2 Read-Only Filesystems
- 8.3 Resource Limits
- 8.4 Security Scanning
- 8.5 Docker Secrets
- 8.6 Production Checklist

CHAPTER 1

Introduction to Docker

Containers, Architecture, Core Concepts & Installation

1.1 What is Docker?

Docker

Docker is an open-source platform for developing, shipping, and running applications inside lightweight, isolated environments called containers. A container packages your application code together with all its dependencies (libraries, runtimes, config) into a single, portable unit that runs consistently on any machine that has Docker installed — from a developer's laptop to a cloud server.

- **Portability** – Build once, run anywhere. Same container runs on Linux, Mac, Windows, AWS, GCP.
- **Isolation** – Containers are isolated from each other and the host; no dependency conflicts.
- **Lightweight** – Containers share the host OS kernel; start in milliseconds vs minutes for VMs.
- **Consistency** – Eliminates 'it works on my machine' – dev, staging, and prod use the same image.
- **Scalability** – Spin up hundreds of containers on a single host or across a cluster.
- **Version Control** – Images are versioned; easy rollback to previous versions.

1.2 VMs vs Containers

Aspect	Virtual Machine (VM)	Docker Container
Isolation	Full OS isolation – hypervisor virtualises hardware	Process-level isolation – shares host OS kernel
Boot Time	Minutes (full OS boot)	Milliseconds (process start)
Size	GBs (full OS + app)	MBs (just app + libs, no kernel)
Overhead	High – each VM has its own OS	Very low – shared kernel
Portability	Heavy – VM images are large	Lightweight – images on any Docker host
Security	Strong isolation (hardware-level)	Moderate – shared kernel (namespaces/cgroups)
Use Case	Run different OS types, strong security requirements	Microservices, CI/CD, app packaging

1.3 Docker Architecture

Docker Engine

Docker Engine is a client-server application. The Docker Daemon (dockerd) runs on the host and manages containers, images, networks, and volumes. The Docker CLI client sends commands to the daemon via REST API. Docker Desktop bundles both for Mac/Windows.

Component	Role
Docker Daemon (dockerd)	Background service that manages all Docker objects (containers, images, networks, volumes)
Docker CLI (docker)	Command-line client; sends API calls to daemon; what you type in terminal
Docker REST API	Interface the client uses to communicate with the daemon
Docker Desktop	GUI application for Mac/Windows; bundles daemon + CLI + Compose
Docker Hub / Registry	Cloud storage for Docker images; Docker Hub is the public default registry
containerd	Low-level container runtime used by Docker daemon to manage container lifecycle
runc	OCI-compliant runtime that actually creates and runs containers

1.4 Core Concepts

Concept	Definition
Image	A read-only template with instructions for creating a container. Like a blueprint or class.
Container	A running instance of an image. Like an object created from a class. Isolated, writable layer.
Dockerfile	A text file with step-by-step instructions to build a Docker image.
Layer	Each Dockerfile instruction creates a read-only layer; layers are shared and cached.
Registry	A repository for storing and distributing Docker images (Docker Hub, ECR, GCR).
Repository	A collection of related images with different tags (e.g. nginx:1.24, nginx:latest).
Tag	A label for a specific image version (nginx:1.24.0, myapp:v2.1, myapp:latest).
Volume	Persistent storage that outlives containers; managed by Docker.
Bind Mount	Maps a host directory/file directly into a container at runtime.
Network	Virtual network connecting containers; enables service-to-service communication.
Compose	Tool for defining and running multi-container applications via a YAML file.

1.5 Installing Docker & Hello World

Installation and first run

```
# ■■ macOS / Windows ██████████  
# Download Docker Desktop from https://www.docker.com/products/docker-desktop  
# Install, start Docker Desktop, verify:  
  
docker --version  
  
# Docker version 27.0.0  
  
# ■■ Ubuntu / Debian Linux ██████████
```

[illegible]

Docker Commands – Complete Reference

2.1 Container Lifecycle Commands

```
# ■■■ docker run ■■■■
```

```
# docker run [OPTIONS] IMAGE [COMMAND] [ARGS]
```

```
docker run nginx # run nginx, attach to output
```

```
docker run -d nginx # -d: detached (background)
```

```
docker run -d --name my-nginx nginx # --name: give container a name
```

```
docker run -d -p 8080:80 nginx # -p HOST:CONTAINER port mapping
```

```
docker run -d -p 8080:80 -p 8443:443 nginx # multiple port mappings
```

```
docker run --rm nginx # --rm: auto-remove when stopped
```

```
docker run -it ubuntu bash # -i: interactive, -t: tty (terminal)
```

```
docker run -d -e MYSQL_ROOT_PASSWORD=secret mysql:8 # -e: env variable
```

```
docker run -d --env-file .env mysql:8 # load env from file
```

```
docker run -d -v myvolume:/var/lib/mysql mysql:8 # -v: volume mount
```

```
docker run -d -v $(pwd)/data:/data nginx # bind mount current dir
```

```
docker run -d --network my-net nginx # attach to custom network
```

```
docker run -d --restart unless-stopped nginx # restart policy
```

```
docker run -d --memory 512m --cpus 1.5 nginx # resource limits
```

```
docker run -d --read-only nginx # read-only filesystem
```

```
docker run -d -u 1001:1001 nginx # run as specific user:group
```

```
# ■■■ Start / Stop / Restart ■■■■
```

```
docker start my-nginx # start stopped container
```

```
docker stop my-nginx # graceful stop (SIGTERM, then SIGKILL after 10s)
```

```
docker stop -t 30 my-nginx # wait 30s before SIGKILL
```

```
docker restart my-nginx # stop then start
```

```
docker kill my-nginx # immediate SIGKILL
```

```
docker pause my-nginx # freeze container processes
```

```
docker unpause my-nginx # unfreeze
```

```
# ■■■ Remove Containers ■■■■
```

```
docker rm my-nginx # remove stopped container
```

```
docker rm -f my-nginx # force remove running container
```

```
docker rm $(docker ps -aq) # remove ALL stopped containers
```

```
docker container prune # remove all stopped containers (with prompt)
```

```
docker container prune -f # without confirmation prompt
```

2.2 Inspecting & Monitoring Containers

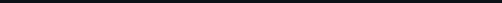
Inspection, logs, stats, events

[illegible]

2.3 Executing Commands Inside Containers

exec, attach, copy, diff, commit

```
# ■■ docker exec - run command in RUNNING container ■■■■■■■■■■
docker exec my-nginx ls /etc/nginx # run non-interactively
docker exec -it my-nginx bash # open interactive shell
docker exec -it my-nginx sh # sh (when bash unavailable)
docker exec -it -e DEBUG=true my-nginx bash # with env variable
```


 Search Docker Hub 

2.5 System & Cleanup Commands

System info, disk usage, cleanup and login

[illegible]

```
docker info # daemon info: containers, images, runtime
```

```
docker version # client and server version
```

```
docker system df # disk usage by images/containers/volumes
```

```
docker system df -v # verbose - per-item breakdown
```

```
# ■■ System Prune (cleanup) ████████████████████████████████████████████████████████████
```

```
docker system prune # remove: stopped containers, unused networks,
```

```
# dangling images, build cache
```

```
docker system prune -a # also removes unused images (not just dangling)
```

```
docker system prune -a --volumes # also removes volumes (CAUTION: data loss!)
```

```
# Individual prune commands
```

```
docker container prune -f # all stopped containers
```

```
docker image prune -a -f # all unused images
```

```
docker volume prune -f # all unused volumes
```

```
docker network prune -f # all unused networks
```

```
docker builder prune -a # build cache
```

[illegible]

```
docker login # login to Docker Hub
```

```
docker login ghcr.io # login to GitHub Container Registry
```

```
docker login registry.example.com # login to private registry
```

```
docker logout # logout
```

CHAPTER 3

Docker Images & Dockerfile

Image Layers, Dockerfile Instructions, Multi-Stage Builds & Best Practices

3.1 Image Layers

Docker Image Layers

A Docker image is built from a stack of read-only layers. Each instruction in a Dockerfile creates a new layer. Layers are cached and shared between images — if you rebuild with only the last layer changed, Docker reuses all previous cached layers. This makes builds very fast and efficient. Only the top container layer is writable.

Image layers visualised

```
# docker history shows each layer and its size
docker history nginx:latest

# IMAGE CREATED CREATED BY SIZE
# 3f8a5c1b9d2e 2 days ago CMD ["nginx" "-g" "daemon off;"] 0B
# 2 days ago COPY nginx.conf /etc/nginx/ 2.3kB
# 2 days ago RUN apt-get install nginx 45.2MB
# 2 days ago RUN apt-get update 28.1MB
# 5 weeks ago FROM debian:bookworm-slim 74.8MB

# Total size = sum of all layers (shared layers are not double-counted)
```

3.2 Dockerfile Instructions – Complete Reference

Instruction	Syntax	Purpose
FROM	FROM image[:tag] [AS name]	Base image; every Dockerfile starts here
RUN	RUN command or RUN ["cmd","arg"]	Execute command during build (creates layer)
COPY	COPY src dest	Copy files from host into image
ADD	ADD src dest	Like COPY but also extracts tarballs, supports URLs
WORKDIR	WORKDIR /path	Set working directory for subsequent instructions
ENV	ENV KEY=VALUE	Set environment variable (persists in container)
ARG	ARG name[=default]	Build-time variable (not in final image)
EXPOSE	EXPOSE port[/protocol]	Document which port the app listens on

Node.js and Python Dockerfiles

3.4 CMD vs ENTRYPOINT

	CMD	ENTRYPOINT
Purpose	Default command (easily overridden)	Main command (harder to override)
Override	docker run image MY-COMMAND replaces CMD	docker run image arg APPENDS to ENTRYPOINT
Usage	Provide default args to ENTRYPOINT	Define the executable that always runs
Example	CMD ["--port","8080"]	ENTRYPOINT ["java","-jar","app.jar"]
Together	CMD provides default args	ENTRYPOINT is the executable, CMD its defaults

CMD vs ENTRYPOINT patterns

```
# ENTRYPOINT + CMD pattern (recommended for CLIs and apps)
ENTRYPOINT ["java", "-jar", "app.jar"]
CMD ["--server.port=8080"] # default port

# Override CMD at runtime:
docker run myapp --server.port=9090 # uses different port

# Override ENTRYPOINT at runtime:
docker run --entrypoint bash myapp # open shell instead of running app

# Shell form vs exec form:
RUN apt-get update # shell form: /bin/sh -c "apt-get update"
RUN ["apt-get","update"] # exec form: direct exec, no shell processing

# Use exec form for CMD and ENTRYPOINT so signals work correctly
```

3.5 .dockerignore

.dockerignore best practices

```
# .dockerignore - exclude files from build context
# Like .gitignore but for Docker builds

# Version control
.git
.gitignore

# Build artifacts (we build inside Docker)
target/
build/
dist/
node_modules/

# IDE files
.idea/
.vscode/
*.iml

# Test files
```


CHAPTER 4

Docker Compose

Multi-Container Applications, compose.yml Reference & Real-World Examples

4.1 What is Docker Compose?

Docker Compose

Docker Compose is a tool for defining and running multi-container Docker applications. You describe your entire stack (app, database, cache, message broker) in a single `compose.yml` file. One command starts, stops, and rebuilds all services together. Compose v2 is built into Docker (`docker compose`) and is the modern standard.

- **Single command** – `docker compose up` starts your entire stack.
- **Service networking** – All services join a default network; communicate by service name.
- **Volume management** – Named volumes persist data across container restarts.
- **Environment isolation** – Each project gets its own network namespace.
- **Dependency ordering** – `depends_on` controls startup order.
- **Scale** – `docker compose up --scale web=3` runs 3 web instances.

4.2 compose.yml Structure Reference

`compose.yml` complete structure reference

```
# compose.yml - Top-level sections
name: my-project # project name (default: directory name)

services: # define application services (containers)
  service-name:
    image: nginx:alpine # use existing image
    build: # OR build from Dockerfile
    context: ./app # build context directory
    dockerfile: Dockerfile.prod
    args:
      BUILD_ENV: production
    container_name: my-container # fixed name (disables scaling)
    restart: unless-stopped # restart policy
    ports:
      - "8080:80" # HOST:CONTAINER
      - "443:443"
    environment:
      - DB_HOST=postgres
      - DB_PORT=5432
```



```

env_file:
- .env # load from file
volumes:
- mydata:/var/lib/data # named volume
- ./config:/etc/app:ro # bind mount, read-only
networks:
- backend
- frontend
depends_on:
postgres:
condition: service_healthy # wait for health check
healthcheck:
test: ["CMD", "wget", "-q0-", "http://localhost:8080/health"]
interval: 30s
timeout: 10s
retries: 3
start_period: 40s
deploy:
resources:
limits:
cpus: "1.0"
memory: 512M
reservations:
memory: 256M
command: ["--config", "/etc/app/config.yaml"] # override CMD
entrypoint: ["/entrypoint.sh"] # override ENTRYPOINT
working_dir: /app
user: "1001:1001"
stdin_open: true # -i (interactive)
tty: true # -t (terminal)
extra_hosts:
- "host.docker.internal:host-gateway"

volumes: # named volumes
mydata: # Docker manages this
pgdata:
driver: local

networks: # custom networks
backend:
driver: bridge
frontend:
driver: bridge

```

4.3 Full Spring Boot + PostgreSQL + Redis Example

Full production stack – Spring Boot + PostgreSQL + Redis + Nginx

[illegible]

[illegible]

.env file for secrets

4.4 Docker Compose Commands

[illegible]

```
docker compose logs --tail 50 # last 50 lines per service
```

[illegible]

```
docker compose exec order-service bash # shell in running container
```

```
docker compose exec postgres psql -U orderuser orderdb # postgres shell
```

```
docker compose run --rm order-service ./migrate # run one-off command
```

[illegible]

```
docker compose build # build all services
```

```
docker compose build order-service # build one service
```

```
docker compose build --no-cache # full rebuild
```

```
# ■■ Config Validation ████████████████████████████████████████████████████████████
```

```
docker compose config # validate and print resolved config
```

```
docker compose config --services # list service names
```

```
docker compose config --volumes # list volume names
```



```
docker run -d -v shared-data:/data --name writer alpine \
sh -c "echo hello > /data/file.txt && sleep 3600"

docker run -d -v shared-data:/data:ro --name reader alpine \
sh -c "cat /data/file.txt"

# :ro makes volume read-only for this container

# Anonymous volume (not named, harder to manage)

docker run -d -v /var/lib/mysql mysql:8

# Auto-named with random hash; cleaned up with docker rm -v
```

5.3 Bind Mounts

Bind Mount

A bind mount maps a specific file or directory on the host machine to a path in the container. Changes on either side are immediately visible. Ideal for development (live code reloading) and sharing config files — but couples the container to the host's directory structure.

Bind mount examples

```
# ■■ Bind mounts in docker run ██████████
```

```
# -v /absolute/host/path:/container/path[:options]
```

```
# Mount current directory (development)
```

```
docker run -d -p 8080:8080 \  
-v $(pwd)/app \  
-w /app \  
node:20-alpine npm start
```

```
# Mount specific config file (read-only)
```

```
docker run -d -v $(pwd)/nginx.conf:/etc/nginx/nginx.conf:ro nginx
```

```
# Mount host directory with specific permissions
```

```
docker run -d \  
--mount type=bind,source=/host/data,target=/app/data,readonly \  
myapp
```

```
# ■■ Bind mounts in compose.yml ██████████
```

```
volumes:  
  
- ./src:/app/src # relative path from compose.yml  
  
- ./config/app.yaml:/app/config/app.yaml:ro  
  
- /var/log/myapp:/app/logs # absolute path
```

```
# ■■ Differences ██████████
```

```
# Volume: docker run -v myvolume:/data <- named volume (Docker managed)  
  
# Bind: docker run -v /host/path:/data <- bind mount (path starts with /)  
  
# Bind: docker run -v ./rel:/data <- bind mount (starts with ./)
```

5.4 tmpfs Mounts

tmpfs mount for sensitive in-memory data

```
# tmpfs: stored in host memory only, never on disk
# Use for sensitive data (passwords, tokens, session data)
```

```
docker run -d \
  --mount type=tmpfs,destination=/app/secret,tmpfs-size=64m \
  myapp

# In compose.yml

services:
  app:
    image: myapp
    tmpfs:
      - /tmp
      - /app/sessions:size=128m
```

5.5 Backup & Restore Volumes

Volume backup and restore patterns

```
# ■■ Backup a volume ████████████████████████████████████████████████████████  
# Spin up temp container, mount volume + host dir, tar the volume  
docker run --rm \  
-v pgdata:/source:ro \  
-v $(pwd)/backup:/backup \  
alpine tar czf /backup/pgdata-backup-$(date +%Y%m%d).tar.gz -C /source .  
  
# ■■ Restore a volume ████████████████████████████████████████████████████████  
docker volume create pgdata-restored  
  
docker run --rm \  
-v pgdata-restored:/target \  
-v $(pwd)/backup:/backup:ro \  
alpine tar xzf /backup/pgdata-backup-20250115.tar.gz -C /target  
  
# ■■ Copy volume contents to another volume ████████████████████████████████  
docker run --rm \  
-v old-volume:/source:ro \  
-v new-volume:/target \  
alpine cp -a /source/. /target/
```


[illegible]

6.3 Port Publishing

Port publishing – all forms

[illegible]

6.4 Network Commands

Network commands, connect/disconnect, troubleshooting

```
# ■■ Create Networks ██████████
```

[illegible]

6.5 Networking in Docker Compose

Compose networking – isolation, aliases, external networks

```
# By default, Compose creates ONE network per project  
# All services join it and can reach each other by service name
```

```
services:
```

```
web:  
  
image: nginx  
  
# web can reach "db" by hostname "db"
```

```
db:  
  
image: postgres  
  
# db NOT exposed to host (no ports:)
```

```
# ■■■ Custom networks for isolation ■■■■
```

[illegible]

Docker Registry

7.1 Docker Hub & Private Registries

```
# ■ Docker Hub
docker login # login to Docker Hub

docker pull nginx:alpine # official image
docker pull myuser/myapp:v1.0 # user/org image

# Push to Docker Hub
docker tag myapp:latest myusername/myapp:v1.0
docker push myusername/myapp:v1.0

# ■ GitHub Container Registry (ghcr.io)
echo $GITHUB_TOKEN | docker login ghcr.io -u USERNAME --password-stdin
docker tag myapp ghcr.io/myorg/myapp:v1.0
docker push ghcr.io/myorg/myapp:v1.0

# ■ Amazon ECR
aws ecr get-login-password --region us-east-1 | \
docker login --username AWS \
--password-stdin 123456789.dkr.ecr.us-east-1.amazonaws.com

docker tag myapp:latest \
123456789.dkr.ecr.us-east-1.amazonaws.com/myapp:v1.0
docker push 123456789.dkr.ecr.us-east-1.amazonaws.com/myapp:v1.0

# ■ Local Private Registry
docker run -d -p 5000:5000 --name registry \
-v registry-data:/var/lib/registry \
--restart unless-stopped \
registry:2

docker tag myapp localhost:5000/myapp:v1.0
docker push localhost:5000/myapp:v1.0
docker pull localhost:5000/myapp:v1.0
```

Docker Security & Best Practices

8.1 Security Best Practices

[illegible]

! Never use 'latest' tag in production. It is mutable and may pull a different image on the next deploy. Always pin to a specific digest or semver tag like nginx:1.25.3.

Quick Reference – Most Used Docker Commands

Command	Purpose
<code>docker run -d -p 8080:80 --name app nginx</code>	Run container detached, port mapping, named
<code>docker run -it ubuntu bash</code>	Interactive terminal inside container
<code>docker run --rm alpine echo hello</code>	One-off container, auto-removed
<code>docker ps / docker ps -a</code>	List running / all containers
<code>docker stop / start / restart name</code>	Control container lifecycle
<code>docker rm -f name</code>	Force remove container
<code>docker logs -f name</code>	Follow container logs
<code>docker exec -it name bash</code>	Shell into running container
<code>docker stats</code>	Live CPU/memory usage
<code>docker inspect name</code>	Full container metadata (JSON)
<code>docker build -t myapp:v1 .</code>	Build image from Dockerfile
<code>docker images / docker image ls</code>	List local images
<code>docker rmi image:tag</code>	Remove local image
<code>docker pull image:tag</code>	Download image from registry
<code>docker push user/image:tag</code>	Upload image to registry
<code>docker system prune -a</code>	Remove all unused containers/images
<code>docker volume create name</code>	Create named volume
<code>docker volume ls / inspect name</code>	List / inspect volumes
<code>docker network create name</code>	Create user-defined network
<code>docker network inspect name</code>	Inspect network and connected containers
<code>docker compose up -d --build</code>	Start all services, rebuild images
<code>docker compose down -v</code>	Stop all and remove volumes
<code>docker compose logs -f service</code>	Follow logs for one service
<code>docker compose exec service bash</code>	Shell into running Compose service
<code>docker compose ps</code>	Status of all Compose services
<code>docker system df</code>	Show Docker disk usage